

✦ JOSA //

Testing, CI/CD, and GitHub Actions

/ Cheap tests, fast feedback, green CI */*

OpenLab Apprenticeship 2026

Week 05

// 1 //

Learning Goals

- Internalize the Test Pyramid and the Testing Trophy as cost/coverage trade-offs.
- Write fast, deterministic tests with clear structure.
- Build a real GitHub Actions workflow.
- Use `act` to run your workflow locally before pushing.

// 2 //

Technical Deep Dive

// 2.1 The Test Pyramid (and the Testing Trophy)

Mike Cohn's Test Pyramid says: have many fast unit tests at the bottom, fewer integration tests in the middle, and very few slow end-to-end (UI) tests at the top. The geometry encodes a cost: a unit test runs in milliseconds; an E2E test can take minutes. Thousands of unit tests are cheaper than dozens of E2E tests.

Kent C. Dodds' [Testing Trophy](#) updates this for modern frontend development: lots of integration tests (because in JS, the boundaries between "unit" and "integration" are fuzzy and integration tests give you the highest confidence per dollar), supported by static analysis (TypeScript, ESLint) at the base.



TIP

The unifying principle: **write the cheapest test that meaningfully exercises the behavior**. A pure function gets a unit test. A function that talks to a database gets an integration test against a real (test) database, not a mocked one. A user-facing flow gets one happy-path E2E test.

// 2.2 Anatomy of a good unit test

Test should generally follow the **Arrange / Act / Assert** structure:

```
def test_parser_handles_trailing_comma():
    # Arrange
    parser = JSON5Parser()
    input_text = '{"a": 1,}'

    # Act
    result = parser.parse(input_text)

    # Assert
    assert result == {"a": 1}
```

Three rules //

1. **One assertion (or one logical assertion) per test.** Multiple assertions hide which one failed.
2. **Names describe the behavior, not the function.** `test_parse` is bad;
`test_parser_handles_trailing_comma` is good.
3. **Tests are deterministic.** No random data unless you use a seeded generator. No real time (`now()`) – inject a clock. No real network – inject a transport.

// 2.3 Fixtures, mocks, and when to use what

- **Fixtures** are pre-built pieces of state your test needs (pytest fixtures, jest setup functions). Use them to provide test data.
- **Mocks** are fake objects that let you assert interactions (e.g., “the email sender was called once with these args”). They’re powerful but coupled – when the real interface changes, every mock breaks.
- **Fakes** are simplified-but-real implementations (an in-memory database). Often better than mocks because they exercise actual logic.



TIP

Heuristic: prefer fakes > mocks. Mock external services you don’t own (Stripe, SendGrid). Don’t mock your own database – run a real one in CI (Postgres in a Docker container is one line of YAML).

// 2.4 A real GitHub Actions workflow

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      fail-fast: false
    matrix:
      os: [ubuntu-latest, macos-latest, windows-latest]
      node: [18, 20, 22]
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node }
          cache: npm

      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage

      - uses: codecov/codecov-action@v4
        with:
          token: ${ secrets.CODECOV_TOKEN }
```

Things to notice //

- **matrix:** runs the same job 9 times (3 OSes × 3 Node versions). Catches “works on my machine” issues in one PR.
- **fail-fast: false** lets all matrix cells run even if one fails – much better signal.
- **cache: npm** lets **setup-node** cache **node_modules** between runs. CI goes from 3 minutes to 30 seconds.
- **secrets.CODECOV_TOKEN** is set in repo settings, never in the YAML. Treat all secrets like passwords.
- **Pinned action versions (@v4).** A future security week will cover why **@main** is dangerous (supply-chain attacks).

// 2.5 Local-first: act

Pushing-to-debug-CI is a special kind of suffering. [act](#) runs your GitHub Actions workflows locally with Docker which saves you the headache:

```
# Install act (Check the GitHub repo for instructions)

# Run all jobs in your default workflow
act

# Run only the test job, with secrets
act -j test --secret-file .secrets
```

It's not a 100% faithful emulator – Windows runners are unsupported, some actions don't behave identically – but for 80% of cases it turns a 10-minute CI feedback loop into a 30-second local one.

// 3 //

Learning Resources

Topic	Resource
The canonical pyramid article	Martin Fowler – The Practical Test Pyramid
Modern frontend take	Kent C. Dodds – The Testing Trophy
GitHub Actions docs	docs.github.com/actions
Local Actions runner	nektos/act
Curated Actions list	Awesome Actions
Property-based testing (Python) - Extra	Hypothesis docs
Property-based testing (JS) - Extra	fast-check
Continuous Delivery	Eric Minick – What is Continuous Delivery? YouTube

// 4 //

Suggested Repositories

- **Any project with a CI badge that's red.** Fixing flaky CI in a real project is one of the most welcome contributions you can make.
- [sdras/awesome-actions](#) – Submit your own github actions workflow or improve docs.

// 5 //

Your Tasks for the Week

1. **Add tests where there are none.** Scout for untested stuff in a GitHub repo with no. Write a test file (or add tests to the existing test file) with well-structured Arrange/Act/Assert tests. Submit a PR.
2. **Build a real workflow.** Take any of your existing repos and add a complete [.github/workflows/ci.yml](#) with: lint, test, matrix on 2+ versions, caching, and a status badge in the README.
3. **Run it with act.** Reproduce your workflow locally with [act](#) before pushing it. Document any differences in your journal.

Soft Skill of the Week: Time Management & Deep Work

Testing, debugging, and CI work require **uninterrupted focus**. A flaky test that takes 90 minutes to track down with concentration takes 4 hours fragmented across Slack pings. Modern software engineering is essentially a fight against fragmentation.

// 6.1 The Two highest-leverage techniques

4. **Time-box.** Decide in advance: “I will spend 90 minutes on this debugging session, then re-evaluate.” Without a box, you’ll either burn out chasing a ghost or give up too early. Use a literal timer. The Pomodoro technique (25 min work / 5 min break) is the classic; many engineers prefer 50/10 or 90/15 for deep work.
5. **One thing at a time.** Multitasking is task-switching, and task-switching is expensive (cognitively, ~25 minutes to fully reload context). Close Slack. Close email. One terminal, one editor, one browser tab on docs. The unease you feel when “missing” notifications is the cost of focus, and it’s worth paying.



TIP

An OSS-relevant practice: batch your code review and async messages. Don’t respond to every PR comment as it lands. Once or twice a day, sit down with all of them, respond to all, push your fixes in one batch. This is also kinder to reviewers — they get one notification cycle, not seven.

// 6.2 Resources

- [Deep Work — Cal Newport.](#)
- [Paul Graham — Maker’s Schedule, Manager’s Schedule](#)